

PetAdopt

A platform that connects animal shelters and pet owners with people who want to adopt pets.

Pages

- 1- Admin dashboard (1- manage users”all users” 2- shelter/owner verification 3- manage users 4- pet posts)**
- 2- Login**
- 3- Register (if shelter/owner sign up he got a message “your account under review ”)**
- 4- Shelter home “1- my pets “add/update/delete” 2- adoption requests 3- profile “see his details and account ” 4- add posts “choose from pets he have ” ”**
- 5- Adopter home “1-browse pets 2-his profile ” (have his previous adoptions , his requests that still not applied or rejected)**
- 6- Favorites**
- 7- Reviews (make reviews in the owners he adopt from only)**

1- Admin (we)

1- Admin can manage (accept or reject) newly registered accounts (Shelters or Pet Owners). -> probably will manage all users (owners , adopter)

2- Admin can approve or reject pet posts before they appear publicly.

2- Shelter / Pet Owner

1-Pet post includes: Owner Name, Pet Name, Age, Breed, Gender, Health Status, Description, Location, and Images.

2-Pet owners can accept or reject adoption requests.

3- Adopter

1- An adopter can browse pets without logging in but cannot apply for adoption unless logged in.

2- Adopters can browse pets and search based on animal type, breed, age, or location.

3- Adopters can submit adoption requests.

4-Adopters can provide a history list of previous adoptions (e.g., past pets, veterinary references, or experience) to be included in their adoption requests.

5- Adopters can save pets to a favorites list.

6-Users who successfully adopt pets can submit reviews and feedback for the shelter or Owner.

Notes

1- You must use real-time sockets as an example. **“SignalR”**

2- DB design (Schema) is required

1- Authentication & User Management -> ()

- Register
- Login
- Roles (Admin , Owner/Shelter , Adopter)
- Account status (Pending / Approved)
- Profile

Backend:

- JWT Authentication
- User table
- Role handling
- Approval logic Of Accounts

Frontend:

- Login page
- Register page
- Profile page

Endpoints

1. POST /api/auth/register

Examples

If owner or adopter

```
{  
  "firstName": "O",  
  "lastName": "m",  
  "email": "o@mail.com",
```

```
"password": "123",  
"role": "adopter/owner"  
"age": 25,  
"location": "Cairo"  
}
```

If shelter

```
{  
  "firstName": "o",  
  "lastName": "m",  
  "email": "shelter@mail.com",  
  "password": "123",  
  "role": "shelter",  
  "shelterName": "shelter 1",  
  "address": "zaid",  
  "description": "We rescue animals",  
  "location": "Cairo"  
}
```

Response if owner or shelter

```
{  
  "success": true,  
  "message": "Account created successfully. Waiting for admin approval.",  
  "data": {  
    "status": "pending"  
  }  
}
```

Response if adopter

```
{  
  "success": true,  
  "message": " account created successfully",  
  "data": {  
    "user": {  
      "id": 1,  
      "role": "owner",
```

```
} }}
```

2. POST /api/auth/login

```
{  
  "email": "o@example.com",  
  "password": "123"  
}
```

Response

```
{  
  "success": true,  
  "message": "Login successful",  
  "data": {  
    "token": "1111111111",  
    "user": {  
      "id": 1,  
      "name": "O",  
      "role": "owner",  
      "status": "approved"}}}
```

Note

If account waiting for approval

```
{  
  "success": false,  
  "message": "Your account is under review"  
}
```

3. POST /api/auth/logout

The Request with Token Only

Response

```
{  
  "success": true,
```

```
"message": "Logged out successfully"
}
```

4. GET /api/auth/profile

Request

Header: Authorization: Bearer token

Response

```
{
  "success": true,
  "data": {
    "id": 1,
    "firstName": "O",
    "lastName": "m",
    "email": "o@example.com",
    "role": "owner",
    "status": "approved",
    "location": "Cairo"
  }
}
```

5. PUT /api/profile

```
{
  "firstName": "Om",
  "lastName": "Wa",
```

```
"phone": "01000000000",  
"location": "Cairo",  
"age": 26,}
```

Response

```
{  
  "success": true,  
  "message": "Profile updated successfully",  
  "data": {  
    "id": 1,  
    "firstName": "Om",  
    "lastName": "Wa",  
    "role": "owner",  
    "location": "Cairo"  
  }  
}
```

6. PUT /api/auth/change-password

```
{  
  "currentPassword": "123",  
  "newPassword": "new123",  
  "confirmPassword": "new123"  
}
```

Response

```
{  
  "success": true,  
  "message": "Password changed successfully"  
}
```

2- Pet Management Feature -> ()

- Add pet
- Edit pet
- Delete pet
- View "My Pets"

Backend:

- Pet entity
- CRUD APIs
- Link pet → owner

Frontend:

- My Pets page
- Add/Edit forms

1. POST /api/pets

```
{  
  "name": "dog",  
  "age": 2,  
  "breed": "Husky",  
  "gender": "male",  
  "healthStatus": "vaccinated",  
  "description": "Friendly dog",  
  "location": "Cairo",  
  "images": [file1, file2]  
}
```


Response

```
{
  "success": true,
  "message": "Pet added successfully. Waiting for admin approval.",
  "data": {
    "id": 1,
    "name": "Max",
    "status": "pending"
  }
}
```

2. PUT /api/pets/{id}

Request

```
{
  "name": "Max Updated",
  "age": 3,
  "breed": "Husky",
  "gender": "male",
  "healthStatus": "vaccinated",
  "description": "Very friendly",
  "location": "Giza"
}
```

Response

```
{
  "success": true,
  "message": "Pet updated successfully",
  "data": {
    "id": 1,
    "name": "Max Updated"
  }
}
```

3. DELETE /api/pets/{id}

Request

(No body)

Response

```
{
  "success": true,
  "message": "Pet deleted successfully"}
```

4. GET /api/pets/my “return my pets”

Request

(Header: JWT)

Response

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "name": "Max",
      "age": 2,
      "breed": "Husky",
      "status": "approved",
      "mainImage": "url"
    },
    {
      "id": 2,
      "name": "Bella",
      "age": 1,
      "breed": "Cat",
      "status": "pending",
      "mainImage": "url"
    }
  ]
}
```

3. Adoption System Feature-> ()

- Send adoption request
- Accept / Reject
- Adoption history

Backend:

- Adoption Request table
- Status (Pending / Approved / Rejected)

Frontend:

- Requests page (Owner/Shelter)
- My Requests (Adopter)

1. POST /api/adoptions “Send Adoption Request”

Request

```
{
  "petId": 5,
  "message": "I have experience raising dogs and a big home."
}
```

Response

```
{
  "success": true,
  "message": "Adoption request sent successfully",
  "data": {
    "id": 10,
    "status": "pending"
  }
}
```

2. GET /api/adoptions/received

“Get Requests for My Pets (Owner/Shelter)”

Response

```
{
  "success": true,
  "data": [
    {
      "id": 10,
      "pet": {
        "id": 5,
        "name": "Max"
      },
      "adopter": {
        "id": 3,
        "name": "Ali"
      },
      "message": "I love dogs",
      "status": "pending",
      "createdAt": "2026-04-12"
    }
  ]
}
```

3. GET /api/adoptions/my

“Get My Requests (Adopter)”

Response

```
{
  "success": true,
  "data": [
    {
      "id": 10,
      "pet": {
        "id": 5,
        "name": "Max"
      },
      "status": "pending"
    },
    {
      "id": 11,
      "pet": {
        "id": 7,
        "name": "Bella"
      },
      "status": "approved"
    }
  ]
}
```

4. PUT /api/adoptions/{id}/accept

Response

```
{
  "success": true,
  "message": "Adoption request approved"
}
```

Note

“ If This request approved , All other requests for same pet rejected ”

5. PUT /api/adoptions/{id}/reject

Response

```
{
  "success": true,
  "message": "Adoption request rejected"
}
```

6. GET /api/adoptions/history

“Adoption History (Adopter)”

Response

```
{
  "success": true,
  "data": [
    {
      "pet": {
        "id": 5,
        "name": "Max"
      },
      "status": "approved",
    }
  ]
}
```

4. Browsing, Favorites & Reviews-> ()

- Browse pets
- Search / filter
- Favorites
- Reviews

Backend:

- Search APIs
- Favorites table
- Reviews table

Frontend:

- Home page
- Pet details page
- Favorites page

1. GET /api/pets “Browse pets”

“if we will search”

GET/api/petstype=dog&breed=husky&location=cairo&minAge=2&maxAge=5&page=1&pageSize=10

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "name": "Max",

```

```

        "type": "dog",
        "breed": "Husky",
        "age": 2,
        "location": "Cairo",
        "image": "url"
    }
],
"pagination": {
    "page": 1,
    "pageSize": 10,
    "total": 100
}
}

```

2. GET /api/pets/{id} ” Pet Details”

Response

```

{
  "success": true,
  "data": {
    "id": 1,
    "name": "Max",
    "age": 2,
    "breed": "Husky",
    "gender": "male",
    "healthStatus": "vaccinated",
    "description": "Friendly dog",
    "location": "Cairo",
    "images": ["url1", "url2"],
    "owner": {
      "id": 5,
      "name": "O"
    }
  }
}

```

3. POST /api/favorites/{petId} “add to favorites”

3. DELETE /api/favorites/{petId} “delete from favorites”

5. GET /api/favorites

Response

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "name": "Max",
      "breed": "Husky",
      "image": "url"
    }
  ]
}
```

6. POST /api/reviews

Request

```
{
  "targetUserId": 5,
  "rating": 5,
  "comment": "Very friendly and honest"
}
```

Response

```
{
  "success": true,
  "message": "Review added successfully"
}
```

7. GET /api/reviews/{userId}

Response

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "reviewer": "Omar",
      "rating": 5,
      "comment": "Great experience",
      "date": "2026-04-12"
    }
  ]
}
```

5. Admin Dashboard + Approval System -> ()

- Approve users (Shelter / Owner)
- Approve pet posts
- Manage all users
- Manage pets

Backend:

- Admin APIs
- Approval logic

Frontend:

- Admin dashboard

1. GET /api/admin/users “Get All Users”

GET /api/admin/users?role=shelter&status=pending-> if i want to filter or select user

Response

```
{
  "success": true,
  "data": [
    {
      "id": 1,
      "name": "Ali",
      "email": "ali@mail.com",
      "role": "shelter",
      "status": "pending"
    }
  ]
}
```

2. GET /api/admin/users/{id} “Get Single User”

3. PUT /api/admin/users/{id}/approve “Approve User”shelter/owne””

Response

```
{
  "success": true,
  "message": "User approved successfully"
}
```

```
}
```

4. PUT /api/admin/users/{id}/reject “Reject User”

Response

```
{
  "success": true,
  "message": "User rejected successfully"
}
```

5. GET /api/admin/pets

GET /api/admin/pets?status=pending&page=1&pageSize=10 -> if i want to filter

Response

```
{
  "success": true,
  "data": [
    {
      "id": 5,
      "name": "Max",
      "owner": "Omar",
      "status": "pending"
    }
  ]
}
```

6. PUT /api/admin/pets/{id}/approve

Response

```
{
  "success": true,
  "message": "Pet approved successfully"
}
```

7. PUT /api/admin/pets/{id}/reject

Response

```
{  
  "success": true,  
  "message": "Pet rejected successfully"  
}
```

8. DELETE /api/admin/pets/{id} “delete pet”

9. DELETE /api/admin/users/{id} “delete user”